

OpenMP

Programacion paralela y concurrente

Angel Ignacio Paredes López

13 de mayo de 2026

¿Qué es OpenMP?

- **Definición:** Es una API diseñada para la programación paralela de memoria compartida.
- **Componentes:**
 - 1 Directivas de compilador (`#pragma omp ...`).
 - 2 Rutinas de biblioteca de tiempo de ejecución (`omp_get_thread_num()`).
 - 3 Variables de entorno (`OMP_NUM_THREADS`).
- **Ventaja:** Permite paralelizar código secuencial de forma incremental.

El Modelo de Ejecución: Fork-Join

- Todo programa OpenMP comienza como un único hilo llamado **Hilo Maestro**.
- Cuando se encuentra una directiva `parallel`, el hilo maestro crea un equipo de hilos (**Fork**).
- Al final de la región paralela, los hilos se sincronizan y terminan, dejando solo al maestro (**Join**).

Master Thread → Parallel Team → Master Thread

Calculo de π secuencial

```
static long num_steps = 100000;
double step;
int main ()
{ int i; double x, pi, sum = 0.0;
  step = 1.0/(double) num_steps;
  for (i=0;i< num_steps; i++){
    x = (i+0.5)*step;
    sum = sum + 4.0/(1.0+x*x);
  }
  pi = step * sum;
}
```

Calculo de π con openmp

```
#include <omp.h>
static long num_steps = 100000; double step;
#define NUM_THREADS 2
void main ()
{ int i, nthreads; double pi, sum[NUM_THREADS];
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds;
    double x;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthreads = nthrds;
    for (i=id, sum[id]=0.0; i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum[id] += 4.0/(1.0+x*x);
    }
  }
}
```

Estrategia del algoritmo:

El patrón de diseño SPMD (Single Program Multiple Data):

- Ejecuta el mismo programa en P elementos de procesamiento, donde P puede ser arbitrariamente grande.
- Utiliza Rank, un identificador que va de 0 a $(P-1)$, para seleccionar entre un conjunto de tareas y para gestionar cualquier estructura de datos compartida.

Resultados:

Hilos	SPMD
1	1.86
2	1.03
3	1.08
4	0.97

¿Porque la mejora no es tan buena?

- Si elementos de datos independientes se encuentran en la misma línea de caché, cada actualización provocará que las líneas de caché se muevan de un hilo a otro. Esto se conoce como "falso compartimiento".
- Si se convierten escalares en un array para admitir la creación de un programa SPMD, los elementos del array son contiguos en memoria y, por lo tanto, comparten líneas de caché. Esto resulta en una escalabilidad deficiente.
- Solución: Rellenar los arrays de forma que los elementos que utilice se encuentren en líneas de caché distintas.

Eliminar el False sharing

```
#include <omp.h>
static long num_steps = 100000; double step;
#define PAD 8 // assume 64 byte L1 cache line size
#define NUM_THREADS 2
void main ()
{ int i, nthreads; double pi, sum[NUM_THREADS][PAD];
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  { int i, id, nthrds;
    double x;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthreads = nthrds;
    for (i=id, sum[id]=0.0; i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum[id][0] += 4.0/(1.0+x*x);
    }
  }
```


Hilos	SPMD	SPMD padded
1	1.86	1.86
2	1.03	1.01
3	1.08	0.69
4	0.97	0.53

¿Realmente necesitamos rellenar nuestros arrays?

- Rellenar arrays requiere un conocimiento profundo de la arquitectura de caché. Si cambias a una equipo con líneas de caché de diferente tamaño, el rendimiento de tu software se verá gravemente afectado.
- Debe haber una mejor manera de lidiar con el falso compartimiento de caché.

¿Cómo interactúan los hilos?

- OpenMP es un modelo de multihilo con direcciones compartidas. Los hilos se comunican compartiendo variables.
- El uso compartido involuntario de datos provoca condiciones de carrera
- Para controlar las condiciones de carrera: Utilice la sincronización para evitar conflictos de datos.
- La sincronización es costosa, por lo que: Modifique la forma en que se accede a los datos para minimizar la necesidad de sincronización.

Sincronización: Barrier

Cada hilo espera hasta que lleguen todos los demás.

```
#pragma omp parallel
{
    int id=omp_get_thread_num();
    A[id] = big_calc1(id);
    #pragma omp barrier
    B[id] = big_calc2(id, A);
}
```

Sincronización: Critical

Mutual exclusion: Solo un hilo a la vez puede entrar en una región crítica.

```
float res;
#pragma omp parallel
{ float B; int i, id, nthrds;
  id = omp_get_thread_num();
  nthrds = omp_get_num_threads();
  for(i=id; i<niters; i+=nthrds){
    B = big_job(i);
    #pragma omp critical
    res += consume (B);
  }
}
```



Sincronización: Atomic

Atomic: Proporciona exclusión mutua, pero solo se aplica a la actualización de una ubicación de memoria (la actualización de X en el siguiente ejemplo).

```
#pragma omp parallel
{
    double tmp, B;
    B = DOIT();
    tmp = big_ugly(B);
    #pragma omp atomic
    X += tmp;
}
```

Ejemplo de implementación de critical

```
void main ()
{ double pi; step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds; double x, sum;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthrds = nthrds;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    for (i=id, sum=0.0; i< num_steps; i=i+nthrds){
      x = (i+0.5)*step;
      sum += 4.0/(1.0+x*x);
    }
    #pragma omp critical
    pi += sum * step;
  }
}
```

El uso inadecuado de critical

```
void main ()
{ double pi; step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds; double x;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthrds = nthrds;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    for (i=id, sum=0.0; i< num_steps; i=i+nthrds){
      x = (i+0.5)*step;
      #pragma omp critical
      pi += 4.0/(1.0+x*x);
    }
  }
  pi *= step;
}
```

Ejemplo de implementación de atomic

```
void main ()
{ double pi; step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds; double x, sum;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthrds = nthrds;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    for (i=id, sum=0.0; i< num_steps; i=i+nthrds){
      x = (i+0.5)*step;
      sum += 4.0/(1.0+x*x);
    }
    sum = sum*step;
    #pragma atomic
    pi += sum ;
  }
}
```


SPMD vs. worksharing

- Una construcción paralela por sí sola crea un programa SPMD es decir, cada hilo ejecuta de forma redundante el mismo código.
- ¿Cómo se dividen las rutas de ejecución del código entre los hilos dentro de un equipo? A esto se le denomina worksharing

Worksharing constructs en un bucle

Divide las iteraciones del bucle entre los hilos de un equipo.

```
#pragma omp parallel
{
    #pragma omp for
    for (I=0; I<N; I++){
        NEAT_STUFF(I);
    }
}
```

Ventajas de utilizar worksharing constructs

Código secuencial

```
for(i=0;i<N;i++) { a[i] = a[i] + b[i];}
```

Código con parallel omp

```
#pragma omp parallel  
{int id, i, Nthrds, istart, iend;  
  id = omp_get_thread_num();  
  Nthrds = omp_get_num_threads();  
  istart = id * N / Nthrds;  
  iend = (id+1) * N / Nthrds;  
  if (id == Nthrds-1)iend = N;  
  for(i=istart;i<iend;i++) { a[i] = a[i] + b[i];}}
```

Parallel omp + worksharing constructs

```
#pragma omp parallel  
#pragma omp for  
for(i=0;i<N;i++) { a[i] = a[i] + b[i];}
```



Cláusulas del planificador (Scheduler)

Las cláusulas de planificación afecta la forma en que las iteraciones del bucle se asignan a los hilos.

- `schedule(static [,chunk])` – Asigna bloques de iteraciones de tamaño “chunk” a cada hilo.
- `schedule(dynamic[,chunk])` – Cada hilo toma “chunk” iteraciones de una cola hasta que se hayan procesado todas.
- `schedule(guided[,chunk])` – Los hilos toman bloques de iteraciones dinámicamente. El tamaño del bloque comienza siendo grande y se reduce hasta el tamaño “chunk” a medida que avanza el cálculo.
- `schedule(runtime)` – La planificación y el tamaño del bloque se toman de la variable de entorno `OMP_SCHEDULE` (o de la biblioteca de tiempo de ejecución).
- `schedule(auto)` – La planificación queda a criterio del tiempo de ejecución (no tiene por qué ser ninguna de las anteriores).

Cláusulas del planificador (Scheduler)

Clausula	Cuando usarla
STATIC	Predeterminado y predecible por el programador
DYNAMIC	Trabajo impredecible y altamente variable por iteración
GUIDED	Caso especial de dinámico para reducir la sobrecarga de planificación
AUTO	Cuando el entorno de ejecución puede “aprender” de ejecuciones anteriores del mismo bucle

Bucles anidados

- Para bucles rectangulares perfectamente anidados, podemos paralelizar varios bucles dentro del nido con la cláusula collapse:

```
#pragma omp parallel for collapse(2)
for (int i=0; i<N; i++) {
    for (int j=0; j<M; j++) {
        . . . . .
    }
}
```

- Se formará un único bucle de longitud $N \times M$ y luego se paralelizará.
- Útil si N es $O(\text{número de hilos})$, ya que paralelizar el bucle exterior dificulta el equilibrio de la carga.

- ¿Cómo se manejaría este caso?

```
double ave=0.0, A[MAX]; int i;  
for (i=0;i< MAX; i++) {  
    ave + = A[i];  
}  
ave = ave/MAX;
```

- Estamos combinando valores en una única variable acumulativa (promedio)... existe una dependencia real entre las iteraciones del bucle que no se puede eliminar fácilmente.
- Esta es una situación muy común... se denomina "reduction".
- La mayoría de los entornos de programación paralela incluyen soporte para operaciones de reducción.

Reduction

- Cláusula de reducción de OpenMP: reduction (op : list)
- Dentro de una estructura paralela:
 - 1 Se crea una copia local de cada variable de la lista y se inicializa según la “op” (p. ej., 0 para “+”).
 - 2 Las actualizaciones se realizan en la copia local.
 - 3 Las copias locales se reducen a un único valor y se combinan con el valor global original.
- Las variables de la “lista” deben compartirse en la región paralela que las contiene.

```
double ave=0.0, A[MAX]; int i;
#pragma omp parallel for reduction (+:ave)
for (i=0;i< MAX; i++) {
    ave + = A[i];
}
ave = ave/MAX;
```


Operadores reduction

Lista de operadores y valores iniciales que se pueden utilizar con reduction

Operador	Valor inicial
+	0
*	1
min	Numero positivo mas grande
max	Numero mas negativo
&	~ 0
	0
^	0
&&	1
	0

Ejemplo de calculo de π con reduction

```
#include <omp.h>
static long num_steps = 100000; double step;
void main ()
{ int i; double x, pi, sum = 0.0;
  step = 1.0/(double) num_steps;
  #pragma omp parallel
  {
    double x;
    #pragma omp for reduction(+:sum)
    for (i=0;i< num_steps; i++){
      x = (i+0.5)*step;
      sum = sum + 4.0/(1.0+x*x);
    }
  }
  pi = step * sum;
}
```

Resultados comparativos finales

Hilos	SPMD	SPMD padded	SPMD critical	reduction
1	1.86	1.86	1.87	1.91
2	1.03	1.01	1.00	1.02
3	1.08	0.69	0.68	0.80
4	0.97	0.53	0.53	0.68